

■ 4.1.3 Recursive Programming

Recursion is an important technique in formulating many kinds of problems. The basic idea is to break your problem up into many equivalent steps. Then your program can simply carry out one of these steps, and call itself “recursively” to have subsequent steps done.

Recursion is often used in languages like C, Pascal and LISP. Until recently, however, standard versions of FORTRAN did not allow recursion. As a result, many algorithms which would most naturally be formulated as recursive programs, were instead implemented as much more complicated non-recursive programs. The algorithms can often be made much clearer and more general by using recursion.

```
BitCount[n_Integer] := Mod[n,2] + BitCount[Quotient[n,2]]
```

```
BitCount[0] = 0
```

A recursive program to calculate the number of ones in the binary decomposition of an integer.

Recursion can take some getting used to. Recursive programs often look deceptively simple. It is hard to imagine how such small programs can do the things they do. The point is that each step may not do very much, but the steps are executed many times. In looking at recursive programs, it is often difficult to see exactly what sequence of steps will be followed when the program runs.

You can think of recursion as just another way to execute commands repeatedly. Any recursive program can always in principle be written using iteration. The recursive version is nevertheless usually much shorter and clearer. The main reasons are that when you use recursion, you do not have to specify the explicit “flow of control”, nor do you have to define so many explicit intermediate variables.

```
log[x_ y_] := log[x] + log[y]
```

A rule that is used recursively to expand out logarithms.

Many kinds of mathematical transformation rules that you specify in *Mathematica* implicitly involve recursion. One example is the rule $\log[x_ y_] := \log[x] + \log[y]$ for expanding out logarithms of products. When you type in $\log[a b c]$, the rule is applied once to give $\log[a] + \log[b c]$. The same rule is then applied again to expand out $\log[b c]$.

In writing a recursive program, you have to provide two things. First, you have to specify the operations that are to be performed at each step. And second, you have to give some kind of “end conditions” that tell the system when to stop taking more steps. In the `BitCount` example above, the first rule specifies how

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

`BitCount[n]` should be calculated in terms of `BitCount[Quotient[n,2]]`. We want this rule to be applied many times, until the argument of `BitCount` is 0. The appropriate end condition in this case is therefore `BitCount[0] = 0`.

When you create a recursive program like `BitCount`, *Mathematica* can usually recognize which rules correspond to special-case end conditions, and which ones are general steps in the recursion. It tries to arrange the special cases first in its list, so that they are checked before the general rules are applied.

There are many mathematical functions that can be computed using “recurrence” or “recursion” relations. One classic example is the factorial function.

```
factorial[n_Integer] := n factorial[n - 1]

factorial[1] = 1
```

The factorial function; a classic example of recursion.

The recursive program for the factorial function is certainly elegant. Nevertheless, on almost all computer systems, it is much less efficient than the obvious iterative program. For problems as simple as computing factorials, it is not difficult to devise iterative procedures, and avoid using recursion. Nevertheless, with more complicated problems, recursion often rapidly becomes overwhelmingly the most convenient approach. There may well be an iterative program that would solve a particular problem. But the complications involved in creating the iterative program may be so great that they would become the predominant factor in formulating the algorithm to be used. It is almost always much better to use a recursive program with a better algorithm, than an iterative program with a worse algorithm.

When you write an iterative program, you explicitly specify which commands are to be executed repeatedly. As a result, it is comparatively easy to foresee how many times your commands will be executed. In a recursion program, on the other hand, the mechanism by which commands are repeated is less explicit. As a result, it is often more difficult to foresee just how much computation will be involved in running a recursive program.

```
fibonacci[n_Integer] := fibonacci[n - 1] + fibonacci[n - 2]

fibonacci[0] = fibonacci[1] = 1
```

The Fibonacci function; another classic recursive program.

Computing the Fibonacci function is another example of a problem that can be formulated elegantly in terms of recursion. It reveals, however, one of the potential

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

pitfalls of recursive programs, that is particularly common with simple functions of integers. The issue is that when you are calculating say `fibonacci[20]`, the value of say `fibonacci[8]` will be needed many times. In fact, you can see that in calculating `fibonacci[n]`, the values of `fibonacci[0]` and `fibonacci[1]` must be needed `fibonacci[n]` times. This means that the time needed to calculate `fibonacci[n]` by this method must grow exponentially with the size of `fibonacci[n]`.

There are several ways to avoid this problem. The clearest one to use in *Mathematica* is to store all the values of `fibonacci[m]` that you calculate.

```
fibonacci[n_Integer] := fibonacci[n] = fibonacci[n - 1] + fibonacci[n - 2]

fibonacci[0] = fibonacci[1] = 1
```

A program for the Fibonacci function that saves the values it has calculated.

If you store the values of `fibonacci` that you have already computed, then you simply have to look them up each time you need them, and you do not have to recompute them. As a result, you can compute the Fibonacci function with very few steps of recursion.

Here is the standard recursive definition of the Fibonacci function, with a piece added to print out the arguments that occur.

```
In[1]:= fib[n_] := (Print[n]; fib[n-1] + fib[n-2])
```

Here are the end conditions for the recursion.

```
In[2]:= fib[0] = fib[1] = 1
Out[2]= 1
```

This computes `fib[6]`, printing out the intermediate cases it uses. Many cases appear several times.

```
In[3]:= fib[6]
6
5
4
3
2
2
3
2
4
3
2
2
Out[3]= 13
```

This defines a Fibonacci function which remembers the values you have computed.

```
In[4]:= mfib[n_] :=
(Print[n]; mfib[n] = mfib[n-1] + mfib[n-2])
```

Here are the end conditions.

```
In[5]:= mfib[0] = mfib[1] = 1
Out[5]= 1
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Many fewer cases are used in computing `mfib[6]`.

```
In[6]:= mfib[6]
6
5
4
3
2
Out[6]= 13
```

`mfib` saves all the intermediate values.

```
In[7]:= ?mfib
mfib

mfib/: mfib[1] = 1
mfib/: mfib[0] = 1
mfib/: mfib[2] = 2
mfib/: mfib[3] = 3
mfib/: mfib[4] = 5
mfib/: mfib[5] = 8
mfib/: mfib[6] = 13
mfib/: mfib[n_] :=
(Print[n]; mfib[n] = mfib[n - 1] + mfib[n - 2])
```

The approach of remembering values you have calculated (sometimes called “dynamic programming”) is a good one in writing programs to evaluate simple integer functions. However, if the arguments to your functions are more general symbolic expressions, the approach is usually not useful, since it is rather rare that the exact same argument occurs many times.

Specifying end conditions for recursion is often more complicated when the arguments to your functions are symbolic expressions. The general principle that *Mathematica* uses is to continue to apply whatever rules you have given until it reaches a “fixed point”, and there are no more rules to apply. A recursive rule like `log[x_ y_] := log[x] + log[y]` has no explicit end condition. The rule stops being applied when none of the logarithms in your expression contain products.

Making sure that there is an appropriate end condition, whether implicit or explicit, is crucial in creating recursive programs. One common way to set up a recursive program is to compute the value of a function with some number of objects in its argument in terms of the same function with fewer objects. You can do this in *Mathematica* by giving definitions like `f[x_ y_] := h[x, f[y]]`. Particularly if you use `x_`, it is very important that you specify the appropriate end conditions. If you do not, *Mathematica* will apply the rules you have given to the case of zero objects forever.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Recursive programs often work by evaluating functions with a sequence of arguments along a particular “track”. You specify end conditions which stop evaluation along that track at some point. Thus, for example, the `factorial` function is evaluated with a sequence of integer arguments, stopping when the end condition at 1 is reached. However, if you try to use the recursive rules for `factorial` with something other than an integer argument, you will get “off the track”, and never run into the end condition that you specified. Instead, *Mathematica* will go on trying to evaluate `factorial` with more and more negative arguments forever. To avoid this kind of thing, it is important to constrain the original arguments to recursive functions so that they do indeed stay on the “track” that leads to the end conditions that you have specified.